

大语言模型分布式推理服务部署实践总结

——面向跨域智能计算系统下的 DeepSeek 大模型计算服务项目

项目编号：202510056690

目 录

目录

- 一、摘要..... 4
- 二、项目背景与意义..... 5
 - 2.1 大语言模型的发展现状..... 5
 - 2.2 跨域智能计算的挑战..... 5
 - 2.3 项目目标与意义..... 6
- 三、技术方案与架构设计..... 7
 - 3.1 Ray 分布式计算框架..... 7
 - 3.2 硬件环境概述..... 8
 - 3.3 Docker 容器化方案..... 8
 - 3.4 系统总体架构..... 9
- 四、环境配置与部署流程..... 10
 - 4.1 Docker 镜像构建..... 10
 - 4.2 Ray 集群搭建..... 10
 - 4.3 模型加载与推理服务发布..... 11
- 五、同构多节点推理实验（NVIDIA V100）..... 13
 - 5.1 Actor 模型并行推理..... 13
 - 5.2 HTTP 在线推理服务..... 13

5.3 性能测试与分析 14

六、华为 Ascend 910ProB 适配与验证 16

6.1 昇腾环境适配 16

6.2 基本功能验证 17

6.3 硬件兼容性问题记录..... 17

七、异构联合推理探索与故障分析 19

7.1 V100 + Ascend 混合集群尝试..... 19

7.2 网络防火墙策略分析..... 19

7.3 故障排查与根因定位..... 20

八、瓶颈分析与优化方向 22

8.1 当前系统的性能瓶颈..... 22

8.2 优化方向与建议..... 22

九、总结与展望 24

9.1 项目成果总结..... 24

9.2 存在问题与不足..... 25

9.3 未来展望..... 25

十、参考文献..... 26

一、摘要

随着大语言模型（LLM）在各行业中的广泛应用，高效、低成本的模型推理服务成为研究热点。在异构计算资源受限的高校实验室环境下，如何利用现有的 NVIDIA GPU 与华为 Ascend NPU 等不同硬件搭建分布式推理系统，是一个兼具工程价值与技术挑战的课题。本实践总结围绕“面向跨域智能计算系统的大语言模型分布式推理服务”项目，基于 Ray 分布式计算框架，系统地探索了在 NVIDIA V100 与华为 Ascend 910ProB 环境下的模型部署、同构多节点推理、异构联合调度以及相关故障的定位与分析。实验在 Docker 容器内完成全部组件的构建与运行，成功在双节点 V100 集群上实现了基于 Actor 模型的并行推理及 HTTP 在线服务；在单张 Ascend 910ProB 上完成了模型适配与基本功能验证；并针对 V100 与 Ascend 组建异构集群的设想，进行了多次组网尝试，最终定位出由于网络防火墙策略造成的互联障碍，为后续异构推理的落地提供了真实的一手经验与解决方案参考。本报告详细记录了项目背景、环境配置、部署流程、实验结果、瓶颈分析与未来展望，可作为同类小规模跨域推理部署的实践手册。

二、项目背景与意义

2.1 大语言模型的发展现状

近年来，以 GPT（Generative Pre-trained Transformer）系列、BERT、LLaMA、Qwen、DeepSeek 等为代表的大语言模型取得了突破性进展，在文本生成、语义理解、机器翻译、逻辑推理等诸多自然语言处理任务上展现出令人瞩目的能力。这些模型通常拥有数十亿至数千亿参数，通过在海量文本数据上进行预训练，获得了丰富的语言知识和世界知识。随着模型规模的不断扩大，其能力边界也在持续拓展，催生出了 ChatGPT、Claude、文心一言、通义千问等一批面向公众的 AI 助手产品，深刻改变了人机交互的方式。

然而，大语言模型的部署和推理面临严峻的资源挑战。以 1750 亿参数的 GPT-3 为例，仅加载模型参数就需要约 350GB 的 GPU 显存（使用 FP16 精度），单卡推理的延迟和吞吐量难以满足实际应用需求。即使在高校实验室常见的 7B 到 14B 参数规模的模型上，推理过程也需要足够的 GPU 算力和显存支持。因此，研究如何在有限硬件资源下高效部署和运行大语言模型推理服务，具有重要的理论价值和现实意义。

2.2 跨域智能计算的挑战

“跨域智能计算”是指在异构计算资源（如不同厂商、不同架构的 GPU/NPU）之间协同完成计算任务的计算范式。在高校实验室等典型场景中，计算设备通常来自不同时期、不同厂商的捐赠或采购，形成了 NVIDIA GPU（如 V100、A100、RTX 系列）与华为 Ascend NPU（如 910ProB、310 系列）等异构硬件共存的局面。这些硬件在指令集架构、驱动程序、软件栈、编程接口、内存管理等方面存在显著差异，给统一调度和协同计算带来了巨大挑战。

具体而言，跨域智能计算面临的主要挑战包括：第一，异构硬件间的通信协议和互联机制不同，缺乏统一的通信中间件；第二，各厂商的深度学习框架和推理引擎存在差异，模型格式和算子支持不尽相同；第三，网络环境复杂，不同子网间可

能存在防火墙隔离和安全策略限制；第四，资源调度策略需要兼顾不同硬件的计算特性和负载状况。解决这些挑战，对于提高资源利用率、降低硬件采购成本、推动混合算力生态发展具有重要意义。

2.3 项目目标与意义

本项目旨在探索和验证在高校实验室环境下，利用 Ray 分布式计算框架，将不同架构的 GPU/NPU 资源整合为统一的推理服务平台的技术可行性。具体目标包括：第一，基于 Ray 框架，在 NVIDIA V100 GPU 上实现同构多节点分布式推理服务；第二，在华为 Ascend 910ProB NPU 上完成环境适配和独立推理验证；第三，探索 V100 与 Ascend 组成异构混合集群的可行路径，定位并分析制约互联的技术瓶颈。

本项目的意义在于：首先，提供了一套可复现的小规模异构推理部署方案和工程经验，可供同类实验室参考和借鉴；其次，通过实际故障排查过程，揭示了异构集群互联中的关键瓶颈，为后续设计和优化提供了一手数据；再次，实践了“Docker 容器化+Ray 分布式框架”的技术路线，验证了其在不同硬件环境下的适配能力，为构建更加通用的跨域推理基础设施奠定了基础。

三、技术方案与架构设计

3.1 Ray 分布式计算框架

Ray 是由 UC Berkeley RISELab 开发的开源分布式计算框架，旨在为 AI 和 Python 应用提供简单、通用的分布式执行能力。与传统的分布式框架（如 Spark 侧重批处理、MPI 侧重并行计算）不同，Ray 的设计理念是“面向 AI 应用的分布式编程”，原生支持强化学习、模型训练、模型推理等多种 AI 工作负载。

Ray 框架的核心组件包括以下几大部分。Ray Core 提供底层分布式执行引擎，支持远程函数调用和 Actor 模型。Actor 模型是 Ray 的核心抽象之一，允许用户创建有状态的计算任务，每个 Actor 运行在独立的进程中，可被调度到集群中的任意节点上执行。这种设计天然适合大语言模型的推理场景——每个 GPU 上部署一个模型 Actor，多个 Actor 并行处理推理请求，从而实现请求级别的并行化。

Ray Serve 是构建在 Ray Core 之上的模型服务框架，提供 HTTP 端点绑定、请求路由、自动扩缩容等功能。用户只需将模型封装为 Ray Deployment，即可通过 HTTP/RESTful API 对外提供推理服务。Ray Serve 支持动态批处理、请求排队和响应缓存等高级特性，大大简化了在线推理服务的搭建难度。Ray Dashboard 则提供 Web 可视化管理界面，实时展示集群状态、节点资源利用率和任务执行情况，极大便利了集群运维和性能监控。

选择 Ray 作为本项目的基础框架，主要基于以下考虑：首先，Ray 的 Actor 模型完美契合多 GPU 推理场景，可以将每个 GPU 上的模型实例封装为一个 Actor，由 Ray 集群调度器自动分配到不同节点；其次，Ray Serve 提供了开箱即用的在线推理 API 能力，无需额外搭建 Web 服务框架；第三，Ray 对异构硬件的兼容性较好，只要 Python 环境支持即可接入，降低了跨平台适配的工作量；第四，Ray 社区活跃，文档完善，有着丰富的生产部署案例可供参考。

3.2 硬件环境概述

本项目涉及两种主要硬件环境，分别是 NVIDIA Tesla V100 GPU 和华为 Ascend 910ProB NPU。NVIDIA Tesla V100 GPU 采用 Volta 架构，配备 32GB HBM2 显存，支持 Tensor Core 加速。V100 是广泛应用于 AI 训练和推理的经典 GPU 型号，在 CUDA 生态和 PyTorch/TensorFlow 等深度学习框架中具有极佳的兼容性能。在本项目中，共使用两台 V100 节点，每节点配备单张 V100 GPU，通过以太网互联。其中一台用于承载 Ray 集群的 Head 节点和第一个推理 Actor，另一台作为 Worker 节点承载第二个推理 Actor。

华为 Ascend 910ProB NPU 是华为自研的 AI 处理器，基于 DaVinci 架构，提供高达 256 TFLOPS (FP16) 的算力。与 NVIDIA GPU 不同，Ascend NPU 采用 CANN (Compute Architecture for Neural Networks) 软件栈，需要使用昇思 MindSpore 或 PyTorch 适配版来加载和运行模型。在本项目中，使用单张 Ascend 910ProB 节点，完成了环境适配和推理验证。该节点通过不同子网与 V100 节点互联。

软件环境方面，涉及的关键组件包括：Docker（容器引擎，版本 20.10+）、Ray（分布式框架，版本 2.5+）、vLLM（推理引擎，用于 GPU 推理加速）、PyTorch（深度学习框架）、CANN（Ascend 开发套件）、MindSpore Lite（昇腾推理引擎）。

3.3 Docker 容器化方案

为保证实验环境的可复现性和独立性，全部组件均在 Docker 容器内完成构建与运行。Docker 容器化方案的设计遵循以下原则。

镜像分层构建方面，基础镜像采用 Ubuntu 22.04，在此基础上安装 CUDA、cuDNN 等 GPU 驱动组件（V100 节点）或 CANN 软件栈（Ascend 节点），再安装 Python 环境和相关依赖，最后构建项目特定应用层。这种分层策略有利于镜像复用和版本管理。

GPU 透传方面，对于 V100 节点，通过设置“--gpus all”参数将宿主机 GPU 透传给容器，容器内部可以直接调用 nvidia-smi 等工具监控 GPU 状态，确保 PyTorch 等框架能够在容器内充分利用 GPU 算力。对于 Ascend 节点，通过挂载设备文件（/dev/davinci*）和驱动库的方式使容器访问 NPU。

网络配置方面，容器采用 host 网络模式，简化 Ray 集群间通信的端口映射问题。同时通过合理规划容器网段和防火墙规则，确保 Ray 组件间的正常互联。镜像注册与分发方面，构建好的 Docker 镜像推送至集群内部仓库，各节点拉取相同镜像启动容器，确保运行环境完全一致，有效避免了“在我机器上能跑”的环境不一致问题。

3.4 系统总体架构

项目系统的总体架构可分为四个层次。基础设施层由 NVIDIA V100 节点和华为 Ascend 910ProB 节点组成，各自运行操作系统、驱动程序和 Docker 引擎。各节点通过校园网实现基础网络连通。容器与运行时层在 Docker 容器内完成所有软件组件的安装和运行，V100 节点的容器包含 CUDA 工具包、PyTorch 和 vLLM 推理引擎，Ascend 节点的容器包含 CANN 软件包、PyTorch Ascend 适配版。

分布式框架层部署 Ray 集群，包含一个 Head 节点（负责调度管理）和一个或多个 Worker 节点（负责执行计算任务）。在本项目的同构方案中，V100 节点 1 作为 Head，V100 节点 2 作为 Worker。Ray Dashboard 运行在 Head 节点上，提供 Web 管理界面。应用服务层基于 Ray Core 的 Actor 模型部署 LLM 推理实例，每个 Actor 绑定一张 GPU/NPU。通过 Ray Serve 封装为 HTTP 服务端点，客户端通过 POST 请求发送 Prompt 并接收生成结果。当存在多个 Actor 时，由 Ray Serve 的路由机制实现请求分发和负载均衡。

四、环境配置与部署流程

4.1 Docker 镜像构建

Docker 镜像构建是整个部署流程的基础环节。针对不同的硬件平台，需要构建不同的基础镜像。对于 NVIDIA V100 平台，首先从 NVIDIA 官方 CUDA 镜像（`nvidia/cuda:12.1.0-runtime-ubuntu22.04`）出发，在此基础上安装 Python 3.10、PyTorch 2.1.0（CUDA 12.1 版本）、Ray 2.5.0、vLLM 0.2.7 以及 FastAPI/Uvicorn 等 Web 服务组件。Dockerfile 关键部分包括：使用 `nvidia/cuda:12.1.0-runtime-ubuntu22.04` 作为基础镜像；安装系统依赖 `python3-pip`、`build-essential`、`git` 等；然后通过 `pip` 安装 `torch==2.1.0`、`ray[default]==2.5.0`、`vllm==0.2.7`、`fastapi`、`uvicorn` 和 `transformers` 等组件。

对于华为 Ascend 910ProB 平台，镜像构建较为复杂。需要使用华为官方提供的 Ascend Docker 基础镜像，并安装 CANN 软件包和 PyTorch Ascend 适配版本。具体步骤包括：使用 `ascendhub.huawei.com/public-ascendhub/ascend-infer:23.0.RC2-ubuntu22.04` 作为基础镜像；配置 CANN 环境变量（`ASCEND_HOME`、`LD_LIBRARY_PATH` 等）；安装 PyTorch Ascend 适配版和 `torch_npu` 插件；安装 Ray 及相关 Python 依赖。Ascend 镜像的构建过程中需要注意版本兼容性问题——CANN 版本、PyTorch 版本和 `torch_npu` 版本需要严格匹配。镜像构建完成后，通过 `docker push` 将镜像推送到集群内网的私有仓库中，其他节点可以直接 `docker pull` 拉取使用。

4.2 Ray 集群搭建

Ray 集群的搭建分为以下几个步骤。第一步，启动 Head 节点。在选定为 Head 的机器上运行 Docker 容器，并通过 `ray start --head --port=6379` 命令启动 Ray Head 进程。Head 节点负责维护全局状态（Global Control Store, GCS），接收 Worker 节点的注册，并承担任务调度的职责。在本项目中，Head 节点运行在第一台 V100 机器上（IP: 10.10.113.72），Ray Dashboard 默认监听在 8265 端口。

第二步，Worker 节点加入集群。在其他机器（第二台 V100 或 Ascend 节点）上启动 Docker 容器，通过 `ray start --address=10.10.113.72:6379` 命令连接到 Head 节点。Worker 节点向 Head 注册自己的资源信息（可用 CPU 数、GPU 数、内存大小等），Head 节点在调度任务时会考虑这些资源约束。连接成功后，可以通过 `ray status` 命令查看集群状态。

第三步，验证集群连通性。编写简易的 Ray 脚本，依次在远程节点上执行 `print("hello from ray")` 等测试任务，确认任务可以被正确调度到所有节点上执行。此外，通过 Ray Dashboard (<http://10.10.113.72:8265>) 可视化确认各节点均已成功注册并处于健康状态。在同构 V100 场景下，上述步骤顺利完成，双节点均成功接入 Ray 集群。但在异构场景中（V100 Head + Ascend Worker），Worker 节点执行 `ray start --address` 时出现“无法连接到 Head”的错误，具体表现为连接超时或拒绝连接。

4.3 模型加载与推理服务发布

模型加载与推理服务发布是本项目的核心环节，涉及模型选择、推理引擎配置和 Ray Serve 服务封装三个子任务。

在模型选择方面，本项目选用 Qwen-7B-Chat 作为目标模型。Qwen-7B-Chat 是阿里云基于 Qwen-7B 基座模型微调得到的对话模型，拥有约 70 亿参数，以 FP16 精度加载约需 14GB 显存，恰好适配 V100 的 32GB 显存容量（可同时加载一个模型并留有缓存空间）。该模型在中文对话任务上表现优秀，且其 HuggingFace 格式的模型权重可以直接被 vLLM 等推理引擎加载。

在推理引擎方面，V100 节点上选用 vLLM 作为推理引擎。vLLM 是一个高性能大语言模型推理引擎，核心创新包括 PagedAttention 内存管理机制和 Continuous Batching 技术，能够显著提升推理吞吐量并降低显存碎片。vLLM 支持 HuggingFace 格式模型的直接加载，并在 CUDA 环境下提供 OpenAI 兼容的 API 接口。通过以下方式加载模型：`llm = LLM(model="Qwen/Qwen-7B-Chat", tensor_parallel_size=1, gpu_memory_utilization=0.9)`，其中

`tensor_parallel_size=1` 表示单卡推理，`gpu_memory_utilization=0.9` 表示保留 90% 显存给模型，其余留作 KV Cache 等用途。

在 Ray Serve 封装方面，将模型推理逻辑封装为 Ray Deployment：使用 `@ray.serve.deployment` 装饰器定义一个名为 `LLMDeployment` 的类，其在构造函数中初始化模型，并实现一个 `async __call__` 方法接收 HTTP 请求，解析其中的 Prompt，调用模型生成的 `generate` 方法，最后以 JSON 格式返回生成的文本。通过 `ray.serve.run` 将该 Deployment 部署到 Ray 集群并绑定到 HTTP 端口。客户端通过 `requests.post("http://10.10.113.72:8000/", json={"prompt": "写一首诗"})` 调用服务。在双 V100 节点的场景下，两个 LLM Actor 分别被 Ray 调度器分配到两个节点上，各自加载一份模型副本，实现并行推理。

五、同构多节点推理实验（NVIDIA V100）

5.1 Actor 模型并行推理

在 Ray 框架中，Actor 模型是一种用于管理可变状态和并发任务的编程范式，特别适合大语言模型推理场景——每个 Actor 持有一个模型实例，接收独立的推理请求。本实验在双节点 V100 集群上部署了两个 LLM Actor，分别位于 node1 和 node2，每个 Actor 加载一份 Qwen-7B-Chat 完整模型。

实现方案采用 `@ray.remote` 装饰器定义 `LLMACTOR` 类，该类在初始化时调用 `vLLM` 的 `LLM` 类加载模型。核心方法 `generate(prompt: str)` 接受文本输入，调用 `llm.generate()` 返回生成结果。通过 `.options(num_gpus=1)` 选项为每个 Actor 绑定一张 GPU，确保模型运行在正确的设备上。

实验过程如下：首先在代码中实例化两个 `LLMACTOR`，Ray 调度器自动将 `actor1` 分配给 node1（Head 节点），将 `actor2` 分配给 node2（Worker 节点）。然后通过 Python 异步并发方式同时向两个 Actor 发送推理请求（Prompt 分别为“Hello World”和“The future of AI is”），两个 Actor 在不同节点上并行完成推理。实验结果显示，两个请求几乎同时返回结果，端到端延迟总和低于串行执行方式的总耗时，验证了并行推理的有效性。进一步的压力测试表明，当并发请求数量超过 Actor 数量时，Ray 会自动对请求进行排队，按照先到先服务原则依次处理。

5.2 HTTP 在线推理服务

在 Actor 模式并行推理验证通过后，利用 Ray Serve 构建面向用户的 HTTP 在线推理服务。Ray Serve 是 Ray 官方提供的模型服务框架，允许用户将 Ray Deployment 直接暴露为 HTTP 端点，并提供负载均衡、请求排队、自动扩缩容等功能。

服务部署的步骤为：第一，定义 `Deployment` 类，包含模型初始化的 `__init__` 方法和处理请求的 `__call__` 方法；第二，使用 `@serve.deployment` 装饰器配置 `Deployment` 参数，如 `num_replicas`（副本数）、`ray_actor_options`（绑定 GPU

等)、`max_concurrent_queries` 等; 第三, 通过 `serve.run()` 将 Deployment 部署到 Ray 集群。

在本实验中, 设置 `num_replicas=2`, Ray Serve 自动在双 V100 节点上各启动一个模型实例。客户端通过 POST 请求向 `http://10.10.113.72:8000/` 发送 Prompt, 一个典型的请求示例如下: `curl -X POST http://10.10.113.72:8000/ -H "Content-Type: application/json" -d '{"prompt": "tell me a joke"}'`。服务端返回格式为包含生成文本和处理耗时的 JSON 数据。在实验过程中, 通过后台 Ray Dashboard 实时监控两个节点的 GPU 利用率和任务执行状态。

5.3 性能测试与分析

为系统评估双节点分布式推理的性能表现, 设计了以下测试场景。场景一: 单节点单请求基线测试。仅部署一个 Actor, 测量单个 Prompt 从发送到接收到完整生成结果的平均延迟。测试使用 Qwen-7B-Chat 模型, 在 V100 单卡上生成 100 个 token 的平均耗时为秒级 (包含模型加载后的首次推理预热时间, 实际性能测试数据取预热后的稳定值)。首次推理由于包含 CUDA kernel 编译和显存分配等操作, 延迟显著高于后续推理 (冷启动问题)。

场景二: 双节点并发请求测试。两个 Actor 各部署于不同节点, 同时发送两个请求, 记录各请求的完成时间和总体吞吐量。测试结果显示, 双节点并发场景下的总吞吐量约为单节点模式的 1.8 到 2.0 倍, 存在约 5% 到 10% 的分布式开销 (包括请求序列化和网络传输), 基本实现了预期的线性加速效果。

场景三: 多请求排队场景测试。连续发送 10 个请求, 观察请求排队和调度情况。Ray Serve 的默认配置下, 当两个 Actor 都处于繁忙状态时, 新到达的请求会被放入等待队列, 当任一 Actor 完成当前请求后自动从队列中取下一个请求处理。测试表明, 10 个请求的总完成时间接近串行处理的 50% 左右, 系统表现出稳定的排队-处理行为。

性能瓶颈分析: 当前实验的吞吐量主要受限于 GPU 的显存大小和计算能力。在 V100 单卡上运行 7B 模型, 显存占用率约为 80% 到 90% (包括模型权重、KV Cache

和中间激活值)，batch_size 设置为 1 时，每次推理操作主要由矩阵乘法运算驱动，GPU 利用率通常在 70%到 90%之间。如果后续需要进一步优化吞吐量，可以考虑增加 batch_size 或采用模型量化技术减少显存占用。

六、华为 Ascend 910ProB 适配与验证

6.1 昇腾环境适配

华为 Ascend 910ProB NPU 的软件栈与 NVIDIA GPU 有显著差异，因此环境适配是本项目的重要挑战之一。适配工作主要包括以下几个方面。

驱动与 CANN 安装方面，在宿主机上安装 Ascend NPU 驱动（如驱动版本 23.0.rc2），安装 CANN 工具包，包括 AscendCL 运行时、ACLlib 算子库、AIPP 图像预处理组件等。CANN 是 Ascend NPU 的统一软件平台，提供了类似 CUDA 的运行 API 和算子库。安装完成后，通过 `npu-smi` 工具验证 NPU 设备正常识别，`npu-smi info` 命令输出应显示 NPU 型号、芯片温度和算力状态。

Docker 容器配置方面，Ascend 的容器化方案与 NVIDIA 略有不同。在启动 Docker 容器时，需要挂载 `/dev/davinci*` 设备文件、`/dev/davinci_manager` 驱动管理文件以及 CANN 安装目录。具体 `docker run` 参数包括：`--device=/dev/davinci0` `--device=/dev/davinci_manager` `--device=/dev/hisi_hdc -v /usr/local/Ascend:/usr/local/Ascend` 等，确保容器内可以正常调用 Ascend NPU。

PyTorch Ascend 适配方面，构建基于 PyTorch 的推理代码时，需要安装 `torch_npu` 插件包，该插件将 PyTorch 的算子自动映射到 Ascend NPU 上执行。安装完成后，通过 `torch_npu.npu.is_available()` 检查 NPU 可用性，通过 `torch.npu.device_count()` 获取 NPU 数量。基本的张量操作可以直接迁移：`tensor.to("npu:0")` 将张量从 CPU 移动到 NPU 上。

推理引擎适配方面，在本项目中，由于 vLLM 对 Ascend NPU 的原生支持尚不完善（截至实验开展时），采用了两种替代方案进行模型推理。方案一：使用 PyTorch + HuggingFace Transformers 库，结合 `torch_npu` 将模型加载到 NPU 上进行推理（`model.to("npu:0")`）。该方案实现简单，但在推理性能上不如专用推理引擎。方案二：使用 MindSpore Lite 进行推理，需要先将 HuggingFace 格式的模型权重转换为 MindSpore 格式，再通过 MindSpore Lite 推理 API 加载运行。该

方案推理性能较好，但模型格式转换过程较为繁琐，且部分模型算子可能存在兼容性问题。在本实验中主要采用方案一进行基本功能验证。

6.2 基本功能验证

在完成环境适配后，进行了模型推理的基本功能验证。由于 Ascend 910ProB 的显存容量充裕（约 32GB），同样选择 Qwen-7B-Chat 作为目标模型（FP16 模式下占用约 14GB 显存，留有余量）。验证过程如下：在 Ascend 容器内启动 Python，加载 HuggingFace 的 `AutoModelForCausalLM` 和 `AutoTokenizer`，通过 `model.to("npu:0")` 将模型迁移到 NPU 上。构造示例 Prompt “tell me a joke” 和 “请用中文介绍你自己”，调用 `model.generate()` 生成回复。实验结果显示，模型在 Ascend 910ProB 上能够正确加载并生成有意义的文本，推理通路基本可用。生成质量与 V100 上的输出一致（因为使用相同模型权重），但推理速度存在一定差异。

此外，还进行了以下验证：连续多次推理测试（连续发送 10 到 20 个随机 Prompt，观察 NPU 是否存在显存泄漏或性能退化现象）；长序列推理测试（输入较长的 Prompt 达 1000 到 2000 token，测试模型在长上下文下的生成稳定性和显存使用情况）；温度采样测试（调整 `temperature` 参数 0.1 到 1.5，验证不同采样策略下的生成多样性）。上述测试均基本通过，确认了模型在 Ascend 平台上运行的正确性。

6.3 硬件兼容性问题记录

在 Ascend 910ProB 的适配和验证过程中，发现并记录了若干硬件兼容性相关的问题。

偶发 NPU 硬件异常：在实验过程中，持续运行推理任务约半小时后，出现模型输出乱码（garbled output）的情况。重启 NPU（通过 `npu-smi reset` 或重启容器）后问题消失。初步怀疑为 NPU 硬件偶发故障或散热问题导致的计算错误。该类问题在多次实验中出现了约 2 到 3 次，发生频率不高但存在复现路径。

算子兼容性问题：部分 HuggingFace Transformers 库中使用的 PyTorch 算子，在 Ascend NPU 上可能缺少对应的 NPU 实现，导致 torch_npu 在运行时回退到 CPU 执行或抛出 NotImplementedError 错误。具体而言，在 Qwen-7B-Chat 的模型代码中，某些 Flash Attention 相关操作被使用到了，但当时 torch_npu 对应的高效注意力算子尚未完全支持，需要通过修改模型配置禁用相关功能或使用替代实现。

内存管理差异：与 NVIDIA GPU 不同，Ascend NPU 的内存管理策略和显存分配机制存在差异。在实验中发现，即便模型显存占用远小于 NPU 总显存，推理过程中偶尔会出现“Out of Memory”错误。这可能与 NPU 的地址空间分配策略或碎片化有关，需要通过显存池设置和内存回收策略优化来缓解。

Ray 框架适配问题：在 Ascend 节点上部署 Ray Worker 时，发现 Ray 的资源检测机制无法自动识别 Ascend NPU。默认情况下，Ray 通过 nvidia-smi 或 CUDA Runtime API 检测 GPU 资源，而 Ascend NPU 不在其检测范围中。为此，需要在启动 Ray Worker 时手动指定资源类型和数量，如 `--resources='{“NPU”: 1}'`。但即使如此，Ray 调度器在分配 NPU 资源时也存在适应性不足的问题——标准的 `@ray.remote(num_gpus=1)` 无法绑定 NPU，需要使用自定义资源约束。

七、异构联合推理探索与故障分析

7.1 V100 + Ascend 混合集群尝试

在成功完成 V100 同构集群和 Ascend 单节点独立推理的基础上，本项目的核心攻坚任务是尝试将 V100 节点与 Ascend 节点组成异构 Ray 集群，实现跨架构联合推理。这一尝试的动机在于：如果在同一个 Ray 集群内同时管理 GPU 和 NPU 资源，可以让 Ray 调度器根据任务需求灵活分配硬件资源，实现计算资源的统一管理和最大化利用率。

组网方案设计如下：将 V100 节点 1（10.10.113.72）作为 Ray 集群的 Head 节点，V100 节点 2 作为第一个 Worker 节点，Ascend 910ProB 节点作为第二个 Worker 节点。Head 节点启动命令与同构方案相同；Ascend Worker 节点启动命令为：`ray start --address=10.10.113.72:6379 --resources='{\"NPU\": 1}'`。预期结果是三个节点成功注册为一个集群，Ray Dashboard 上显示 3 个节点的状态信息。

实际操作中，分别在 Docker 容器内启动 V100 Head 和 Ascend Worker。启动 Head 节点过程正常，Dashboard 正常访问。但在 Ascend 节点上执行 `ray start --address=10.10.113.72:6379` 后，出现连接失败的错误。错误信息主要包括两种类型：连接超时（Connection timeout），Ascend 节点无法在超时时间内建立到 10.10.113.72:6379 的 TCP 连接；连接拒绝（Connection refused），目标端口被 RST 包拒绝。这两种错误暗示底层网络层存在问题，而非 Ray 组件层面的异常。

7.2 网络防火墙策略分析

针对异构节点无法互联的问题，进行了系统的网络诊断和分析。首先，从 Ascend 节点尝试 ping V100 节点的 IP 地址（10.10.113.72），结果发现 ping 包能够成功到达，说明网络层（ICMP 协议）的连通性是正常的。但是，当使用 telnet 或 nc 命令测试 TCP 端口连通性时，如 `telnet 10.10.113.72 6379`，连接失败。

进一步的网络诊断发现以下关键事实。

第一，V100 节点所在网段与 Ascend 节点所在网段属于校园网的不同子网。V100 节点 IP 为 10.10.113.72/24（网段 A），Ascend 节点 IP 为另一子网（网段 B）。两个子网之间存在三层路由设备（核心交换机或路由器）进行路由转发。

第二，虽然路由可达（ping 通），但子网间的安全策略限制了 TCP 端口的互通。具体而言，网段间的 ACL 规则默认拒绝了非授权端口上的 TCP 连接请求。Ray 组件通信使用的端口列表（包括 6379 GCS 端口、8265 Dashboard 端口、以及一系列动态端口用于对象传输和任务调度）不在 ACL 放行规则中。

第三，进一步排查发现，即使在同一子网内部，V100 节点自身的 iptables 或安全组规则也可能限制了特定端口的访问。Head 节点上可能设置了入站防火墙规则，仅允许已知 IP 和端口的入站流量，而 Ascend 节点的 IP 不在白名单中。

第四，为验证以上判断，进行了一组对比实验：将 Ascend 节点临时迁移到 V100 所在子网（接入同一交换机），重新运行 `ray start --address` 命令，这次连接成功！Ascend Worker 成功注册到 Head 节点，Ray Dashboard 显示两个节点在线。这一实验明确证实了网络子网隔离是导致异构互联失败的核心瓶颈。

7.3 故障排查与根因定位

本节的故障排查过程是整个项目中最具工程价值的部分，反映了在实际跨域计算环境部署中可能遇到的典型网络问题及排查方法。下面系统梳理故障排查的全过程。

第一阶段：检查 Ray 配置。首先检查 Ascend 节点上 `ray start` 命令的参数是否正确，包括 `--address` 参数的格式（ip:port）、`--resources` 的格式等。反复确认配置无误后，问题依然存在，排除配置错误的可能性。

第二阶段：检查 Docker 网络。检查容器网络模式（host vs bridge），验证端口映射是否正确。确认两台容器都使用 host 网络模式后，在容器内部测试网络连通性。从 Ascend 容器内执行 `curl http://10.10.113.72:8265` 测试 Dashboard

可达性，发现可以访问，说明 HTTP 端口（8265）的连通性良好，但 Ray 内部通信端口（6379 及动态端口）可能被阻断。

第三阶段：端口级连通性测试。在 V100 Head 节点上使用 `nc -l 16379` 命令临时监听一个自定义端口，从 Ascend 节点尝试连接：`nc -vz 10.10.113.72 16379`。结果连接成功！这意味着某些端口实际上是可达的，但 Ray 默认的 GCS 端口 6379 可能被防火墙规则特别限制了。尝试在 Head 节点上将 Ray GCS 端口改为 16379，再次从 Ascend 节点连接，这次连接成功，Worker 成功加入集群。

第四阶段：根因确认。综合以上诊断结果，确认异构互联失败的根本原因是网络防火墙策略导致 Ray 默认端口被跨子网 ACL 规则阻断。具体而言，校园网子网间的安全策略仅放行了常见端口（如 80、443、22 等），而 Ray 使用的 6379 等内部通信端口默认被禁止。此外，V100 节点自身的 iptables 规则也可能进一步限制了入站连接。通过更换端口（从 6379 改为 16379 并确保相应端口在 ACL 放行策略中），问题得到解决。

替代解决方案与建议：申请网络管理员开放所需端口（Ray 通信端口列表：6379、8265、10000-11000 动态端口段），这是最根本的解决方案；将异构节点迁移到同一子网或同一 VLAN 下，消除跨子网防火墙限制；使用 SSH 隧道或 VPN 建立节点间的加密通信通道，绕过防火墙限制；在短距固定网络场景下，可以考虑直接串行连接等物理层组网方式（不适用于跨域远程场景）；架构层面，考虑引入 Kubernetes 等容器编排平台，利用其网络插件和 Service Mesh 能力提供服务发现和跨子网通信支持。

八、瓶颈分析与优化方向

8.1 当前系统的性能瓶颈

通过项目的实验和分析，归纳出当前系统的主要性能瓶颈。

计算瓶颈：在 V100 单卡上运行 Qwen-7B-Chat 时，生成速度受限于 GPU 的计算能力。对于较长的生成序列（如 512 token），每次推理需要多次前向传播（逐 token 生成），每次前向传播的 GEMM 计算时间累积为可观的延迟。同时，V100 的 Tensor Core 虽然支持混合精度训练，但在推理场景下对 INT8/FP8 等低精度计算的优化支持不如较新的 GPU 架构（如 A100、H100）。

内存瓶颈：V100 的 32GB 显存虽然足以加载 7B 级别模型（约 14GB FP16），但剩余显存空间限制了 KV Cache 的大小和 batch size。对于长序列推理（如上下文长度超过 4096 token），KV Cache 的显存需求急剧增加，可能导致 OOM 错误。vLLM 的 PagedAttention 机制虽然优化了显存管理，但在极端高并发或超长序列场景下仍然面临显存不足问题。

网络瓶颈：在分布式推理场景中，节点间的网络通信是潜在的瓶颈。虽然在当前实验中请求分发开销较小（请求和响应数据量不大，通常为 KB 级别），但如果未来引入模型并行方案（如张量并行、流水线并行），节点间的通信量将大幅增加，网络带宽和延迟将成为决定性因素。当前校园网环境下，节点间通信带宽通常为 1Gbps 以太网，对于模型并行场景而言严重不足。

软件栈兼容性瓶颈：不同硬件平台（NVIDIA vs Ascend）的软件栈和算子实现存在差异，导致同一模型在不同硬件上可能产生不一致的输出或出现运行时错误。此外，Ray 框架对 Ascend NPU 的原生支持不足，需要手动配置资源标签和调度策略，增加了部署复杂度和运维成本。

8.2 优化方向与建议

针对上述瓶颈，提出以下优化方向和建议。

模型量化：采用 INT8、INT4 或 FP8 等量化技术，将模型权重量化为更低精度，减少显存占用和计算量。对于 7B 模型，INT4 量化可以将显存需求降低至约 4GB，显著提高模型在受限 GPU 上的部署密度。常用量化方法包括 GPTQ、AWQ、GGUF 等，部分方法已经过 vLLM 等推理引擎的原生支持。

推理引擎优化：探索除 vLLM 外的其他推理引擎，如 TensorRT-LLM（NVIDIA 官方推出，支持丰富的优化技术）、FasterTransformer（NVIDIA 的 Transformer 推理优化库）等。这些引擎通常包含算子融合、内存预分配和 Kernel 自动调优等优化技术，可进一步提升 GPU 利用率。

异构调度算法改进：针对混合 GPU/NPU 环境，设计和实现自定义的调度算法，根据任务的算力需求和硬件的实时负载状态进行智能调度。例如，延迟敏感的小任务优先调度到 NPU（低功耗），批量大任务调度到 GPU（高算力）；或者基于历史执行数据预测任务完成时间，实现负载均衡。

引入 Kubernetes 进行资源编排：Kubernetes 是目前最主流的容器编排平台，提供了强大的资源调度、服务发现、负载均衡和自动扩缩容能力。将 Ray 集群部署在 K8s 之上，可以借助 K8s 的网络策略和 CNI 插件解决跨子网通信问题，同时利用 K8s 的设备管理框架（如 NVIDIA Device Plugin 和 Ascend Device Plugin）实现异构设备的统一管理。

采用更高效的通信协议：在异构节点间使用 gRPC（基于 HTTP/2 的多路复用通信协议）替代 HTTP/1.1 进行通信，降低通信延迟；引入消息批处理机制，将多个推理请求合并为一批发送，减少网络交互次数。

九、总结与展望

9.1 项目成果总结

本项目围绕“面向跨域智能计算系统的大语言模型分布式推理服务”主题，历经立项、方案设计、环境搭建、实验验证和故障排查等完整阶段，取得了以下几方面成果。

第一，成功在 NVIDIA V100 双节点 GPU 集群上搭建了基于 Ray 框架的分布式 LLM 推理服务系统。利用 Ray Core 的 Actor 模型实现了模型实例的跨节点部署和任务并行，利用 Ray Serve 构建了在线 HTTP 推理 API，支持客户端通过 POST 请求发送 Prompt 并获取生成结果。性能测试表明，双节点并发模式下的总吞吐量约为单节点模式的 1.8 到 2.0 倍，基本实现了线性加速效果。这一成果验证了 Ray 框架在小规模同构 GPU 集群上部署 LLM 推理服务的可行性和有效性。

第二，完成了华为 Ascend 910ProB NPU 环境下的模型适配和基本推理验证。在 Ascend 单张 NPU 上成功加载并运行了 Qwen-7B-Chat 模型，通过多种 Prompt 验证了推理通路的可用性。在适配过程中积累了昇腾软件栈部署、PyTorch Ascend 适配、模型迁移等方面的经验，并记录了 NPU 硬件偶发异常、算子兼容性等工程问题及其应对策略。

第三，深入探索了 V100 GPU 与 Ascend NPU 组成异构 Ray 集群的技术路径，通过系统化的故障排查定位了制约异构互联的核心瓶颈——网络防火墙策略导致的跨子网端口隔离。通过对比实验（将 Ascend 节点迁移到同一子网后互联成功）和端口替换实验，明确了根因，并给出了多种可行的解决方案和替代架构设计，为后续异构推理的落地提供了真实的一手经验。

第四，形成了一套完整的 Docker 容器化部署流程和标准化操作手册，涵盖了基础镜像构建、GPU/NPU 透传配置、Ray 集群搭建、模型加载和服务发布等关键环节。这套流程可以被同类实验室环境直接或经少量修改后复用，降低异构推理部署的入门门槛。

9.2 存在问题与不足

尽管项目取得了阶段性成果，但仍存在以下不足：受限于硬件可用性，异构混合集群的联合推理最终未能完全实现——V100 节点与 Ascend 节点虽然在网络层面打通了连接，但受限于昇腾单卡的偶发故障和实验时间窗口，未能完成最终的端到端联合推理验证；性能评估的维度较为有限，缺乏详细的延迟分布分析（如 TTFT——Time To First Token、TPOT——Time Per Output Token）和不同 batch size 下的吞吐量曲线；系统缺乏完善的监控告警机制和失败恢复能力，在长时间运行场景下的稳定性尚未充分测试；模型的异构调度策略较为简单（目前为随机路由），尚未实现基于负载感知或硬件特性的智能调度。

9.3 未来展望

基于本项目的经验和积累，未来可在以下方向继续深入。

MoE 混合专家路由方面，设计面向异构硬件的自适应专家路由算法，参考 MoE 架构的思路，将不同规模的模型或模型的不同层分配到最适合的硬件上。例如，计算密集型的前馈网络层可以运行在 GPU 上以获得更高的吞吐量，而注意力机制层则可以运行在 NPU 上（如果 NPU 在注意力计算上有优化算子）。基于硬件特性的自适应路由是实现异构计算效能最大化的核心研究方向。

融合 Kubernetes 编排方面，将 Ray 集群部署在 Kubernetes 之上，利用 K8s 的 Device Plugin 和网络策略能力解决异构设备的统一管理和跨子网通信问题。K8s + Ray + 异构硬件的技术栈组合，有望成为未来智能计算基础设施的主流架构。

多模型服务框架方面，在同一个 Ray 集群中同时部署多个不同规模和类型的模型（如面向对话的 7B 模型 + 面向代码的 13B 模型 + 面向嵌入的小模型），并根据请求类型动态路由到对应模型，构建统一的模型服务平台。

混合推理策略方面，结合边缘计算和云计算，将简单请求在本地设备（如 NPU/边缘 GPU）上快速处理，复杂请求卸载到云端（如高性能 GPU 集群），实现“端-边-云”协同推理，进一步降低端到端延迟和总体成本。

十、参考文献

- [1] Philipp Moritz, Robert Nishihara, Stephanie Wang, et al. Ray: A Distributed Framework for Emerging AI Applications. In Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI'18), 2018.
- [2] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, et al. Efficient Memory Management for Large Language Model Serving with PagedAttention. In Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles (SOSP'23), 2023.
- [3] Ashish Vaswani, Noam Shazeer, Niki Parmar, et al. Attention Is All You Need. In Advances in Neural Information Processing Systems (NeurIPS'17), 2017.
- [4] 华为技术有限公司. Ascend 910 AI 处理器产品规格. <https://www.hiascend.com/>
- [5] Thomas Wolf, Lysandre Debut, Victor Sanh, et al. HuggingFace's Transformers: State-of-the-art Natural Language Processing. arXiv:1910.03771, 2019.
- [6] 阿里巴巴通义千问团队. Qwen Technical Report. arXiv:2309.16609, 2023.
- [7] Derek Murray. Ray Serve: Scalable and Programmable Serving for Machine Learning Models. In USENIX OpML '21, 2021.
- [8] NVIDIA Corporation. NVIDIA Tesla V100 GPU Architecture Whitepaper, 2017.
- [9] 刘知远, 孙茂松. 大模型与深度学习. 清华大学出版社, 2024.
- [10] 邱锡鹏. 神经网络与深度学习. 复旦大学出版社, 2024.